

Programmation et Algorithmique I

Corrigé du test du 3 novembre 2010

Remarque

Ce document contient *une* solution pour chaque problème du test de novembre. Bien sûr, les algorithmes pour résoudre un problème ne sont pas uniques et votre algorithme, s'il est différent, est peut-être correct également.

Problème 1 : complexité

Rappel de la fonction `f` : elle prend une liste `t` en entrée et retourne un nombre entier

```
1 def f(t):
2     if est_acceptee(len(t)):
3         return reduit(t, 1)
4     else:
5         sum = 0
6         for item in t:
7             sum += reduit(t, item)
8         return sum
```

Notons $T_1(k)$ le temps d'exécution de `est_accepte(k)` (où k est l'entier passé en paramètre de la fonction) et $T_2(m)$ le temps d'exécution de `reduit(liste, p)` où m est le nombre d'éléments dans la liste `liste`. Pour répondre aux 3 sous-questions facilement, on calcule d'abord la complexité de `f(t)` d'une manière générale. Dans l'énoncé, il est demandé d'exprimer cette complexité en fonction du nombre n d'éléments de la liste `t`.

Cette fonction est composée d'une instruction conditionnelle à deux branches. Evaluons les temps d'exécution de la condition et du corps des deux branches. Le temps d'exécution du test de la ligne 2 est en $\mathcal{O}(T_1(n))$ puisque le paramètre passé à la fonction `est_accepte` est `len(t)`, qui est égal à n . Le corps de la branche `if` (ligne 3) est réalisé en un temps d'exécution $\mathcal{O}(T_2(n))$ puisque la liste `t` passée en paramètre de `reduit` est de longueur n . Enfin, le corps de la branche `else` contient une boucle `for` itérant sur la liste `t` : elle est donc exécutée n fois et son corps est exécuté en un temps $T_2(n)$ (car appel à `reduit` sur `t`). Le temps total pour la branche `else` est donc en $\mathcal{O}(n \cdot T_2(n))$ car les autres instructions (lignes 5 et 8) sont en $\mathcal{O}(1)$.

Il reste donc à appliquer la règle utilisée pour les instructions conditionnelles pour conclure que la complexité de `f(t)` est

$$\mathcal{O}(T_1(n)) + \max(\mathcal{O}(T_2), \mathcal{O}(n \cdot T_2(n))).$$

Comme $n \geq 1$, le coût de la branche **else** sera toujours plus grand ou égal au coût de la branche **if** et on peut simplifier cette expression en $\mathcal{O}(T_1(n) + n \cdot T_2(n))$.

- a) Dans ce cas $T_1(n) = \mathcal{O}(1)$ et $T_2(n) = \mathcal{O}(1)$. Il s'ensuit que **f** est en $\mathcal{O}(n)$.
- b) Dans ce cas $T_1(n) = \mathcal{O}(1)$ et $T_2(n) = \mathcal{O}(n)$. Il s'ensuit que **f** est en $\mathcal{O}(n^2)$.
- c) Dans ce cas $T_1(n) = \mathcal{O}(n^2)$ et $T_2(n) = \mathcal{O}(\sqrt{n})$. Il s'ensuit que **f** est en $\mathcal{O}(n^2 + n \cdot \sqrt{n}) = \mathcal{O}(n^2)$.

Erreur très fréquente : beaucoup d'étudiants n'ont pas remplacé k et m par n dans leurs raisonnements. Les notations k et m servaient à définir les temps T_1 et T_2 , mais dans la fonction **f**, on a $k = m = n$.

Problème 2 : Exactitude

- a) Pour prouver que la fonction s'arrête, il faut prouver que la boucle est itérée un nombre fini de fois. La condition d'arrêt (négation de la condition de la boucle) nous dit que la boucle s'arrêtera quand $i \geq n$. Or, n est un entier positif (représente la longueur de la liste entrée en paramètre) et i est initialisé à 0. Dans le corps de la boucle, i est incrémenté de 1 à chaque itération. Il s'ensuit que i atteindra la valeur de n en un nombre fini d'itérations.

Note : On peut le prouver également par récurrence.

- b) L'assertion (b) n'est pas un invariant, car, lors du dernier tour de boucle, la variable i prend la valeur n , ce que n'autorise pas l'assertion (b). Elle est donc fausse juste avant de sortir de la boucle. L'assertion (c) n'est pas un invariant de la boucle car elle affirme que *somme* contient à tout moment la somme des éléments se trouvant dans les cases de 1 à i . Or, ceci est faux dès l'entrée dans la boucle. En effet, en entrant dans la boucle, on a $i = 1$ et *somme* = 0 (on a encore rien ajouté dans *somme*), alors que d'après l'assertion (c), on devrait avoir *somme* = $\sum_{j=0}^0 \text{liste}[j] = \text{liste}[0]$. Ce n'est clairement pas le cas. Prouvons maintenant que l'assertion (a) est un invariant de la boucle.

Pour le prouver, il faut montrer deux choses : (i) l'invariant de boucle est respecté *avant* l'exécution de la boucle ; (ii) si l'invariant est vrai après la k ème itération, alors il est également vrai après la $(k + 1)$ ème itération.

Notations utilisées ci-dessous : en logique, $\wedge$ signifie "et" (conjonction) ; et $\neg$ signifie "non" (négation).

- (i) Comme *somme* et i sont initialisés à 0, l'invariant de boucle devient

$$0 \leq n \wedge 0 = \sum_{j=0}^{0-1} \text{liste}[j]$$

$\Leftrightarrow$

$$0 \leq n \wedge 0 = 0$$

qui est trivialement vrai (n est un entier positif).

- (ii) Seules les valeurs de i et de *somme* sont modifiées dans le corps de la boucle. On note i_{old} et *somme_{old}* les valeurs de i et *somme* après la k ème itération ($k \geq 0$). On suppose l'invariant de boucle vrai après l'exécution de la k ème itération, c'est à dire que

$$i_{old} \leq n \quad \wedge \quad \text{somme}_{old} = \sum_{j=0}^{i_{old}-1} \text{liste}[j] \tag{1}$$

est vrai. Notons i_{new} et *somme_{new}* les valeurs de i et *somme* après la $(k + 1)$ ème itération et prouvons que l'invariant est toujours vrai après l'exécution de celle-ci, c'est à dire que

$$i_{new} \leq n \wedge \text{somme}_{new} = \sum_{j=0}^{i_{new}-1} \text{liste}[j].$$

La première instruction du corps de la boucle donne

$$somme_{new} = somme_{old} + liste[i_{old}] \quad (2)$$

et la seconde nous donne

$$i_{new} = i_{old} + 1 \quad (3)$$

De plus, le fait que la $(k+1)$ ème soit exécutée nous indique que la condition de la boucle a été évaluée vraie après la k ème itération de la boucle. Donc nous avons

$$i_{old} < n \quad (4)$$

En substituant i_{old} et $somme_{old}$ (grâce à (1) et (3)) dans (2), on obtient

$$\begin{aligned} somme_{new} &= \sum_{j=0}^{i_{old}-1} liste[j] + liste[i_{old}] \\ \Leftrightarrow somme_{new} &= \sum_{j=0}^{i_{new}-2} liste[j] + liste[i_{new} - 1] \\ \Leftrightarrow somme_{new} &= \sum_{j=0}^{i_{new}-1} liste[j]. \end{aligned}$$

En substituant i_{old} ((grâce à (3)) dans (4), on obtient

$$i_{new} \leq n.$$

L'invariant est donc vrai après la $(k+1)$ ème itération.

- 3) Pour prouver que l'algorithme est correct, on procède de façon classique, en commençant par calculer la négation de la condition de la boucle :

$$\neg C \equiv i \geq n$$

En prenant la conjonction avec l'invariant prouvé au point précédent, on obtient une assertion qui est vraie à la fin de la boucle :

$$\begin{aligned} \neg C \wedge Inv &\equiv i \geq n \wedge i \leq n \wedge somme = \sum_{j=0}^{i-1} liste[j] \\ &\equiv i = n \wedge somme = \sum_{j=0}^{i-1} liste[j] \\ &\equiv somme = \sum_{j=0}^{n-1} liste[j] \end{aligned}$$

ce qui prouve que la boucle calcule bien la somme des n éléments de la liste.

Problème 3 : Nombres premiers

```
import math

def est_premier(candidat, premiers):
    if candidat < 2 :
        return False
    racine = math.sqrt(candidat)
    indice=0
    while indice < len(premiers) and premiers[indice] <= racine:
        if candidat % premiers[indice] == 0:
            return False
        indice+=1
    return True

def generer_nombres_premiers(nb):
```

```

if nb == 0:
    return []
liste = [2]
candidat = 3
while len(liste) < nb:
    if est_premier(candidat, liste):
        liste.append(candidat)
        candidat+=2
return liste

```

Problème 4 : Opérations matricielles

```

def add(m1, m2):
    if len(m1) == len(m2) and len(m1[0]) == len(m2[0]):
        m3 = []
        for i in range(len(m1)):
            m3_line = []
            for j in range(len(m1[0])):
                m3_line.append(m1[i][j] + m2[i][j])
            m3.append(m3_line)
        return m3

def multiply(m1, m2):
    if len(m1[0]) == len(m2):
        m3 = []
        for i in range(len(m1)):
            m3_line = []
            for j in range(len(m2[0])):
                m3_case = 0
                for k in range(len(m1[0])):
                    m3_case += m1[i][k] * m2[k][j]
                m3_line.append(m3_case)
            m3.append(m3_line)
        return m3

```

Problème 5 : Chaînes de caractères et récursivité

```

def contient(mot, caractere):
    if len(mot)==0:
        return False
    elif mot[0]==caractere:
        return True
    else:
        return contient(mot[1:],caractere)

def test_contient():
    assert contient('abcd','a')
    assert contient('abcd','d')
    assert not contient('', 'a')

```

```

    assert not contient('abcd','e')

####

def palindrome(mot):
    if len(mot)<2:
        return True
    else:
        if mot[0]==mot[-1]:
            return palindrome(mot[1:-1])
        else:
            return False

def test_palindrome():
    assert palindrome('')
    assert palindrome('a')
    assert palindrome('aa')
    assert palindrome('aba')
    assert palindrome('abba')
    assert not palindrome('abb')
    assert not palindrome('bba')

####

def longueur_paire(mot):
    if mot=='':
        return True
    elif mot[1:]=='':
        return False
    else:
        return longueur_paire(mot[2:])

def test_longueur_paire():
    assert longueur_paire('')
    assert longueur_paire('aa')
    assert longueur_paire('abcd')
    assert not longueur_paire(' ')
    assert not longueur_paire('a')
    assert not longueur_paire('abc')

####

def test_all():
    test_contient()
    test_palindrome()
    test_longueur_paire()
    print 'ok'

test_all()

```